Exercises

1-Write parameterized macros that compute the following values.

(a) The cube of x.
(b) The remainder when n is divided by 4.
(c) 1 if the product of x and y is less than 100, 0 otherwise.
(a) The cube of x. (b) The remainder when n is divided by 4. (c) 1 if the product of x and y is less than 100, 0 otherwise.

answer:

(a):

```
#define cubs(x) ((x)*(x)*(x))
```

Side effects causes undefined behavior:

```
int i = 2;
int r = cube(i++);
// → ((i++)*(i++)*(i++))
```

Overflow

```
cube(2000000)   // 2000000³ = 8 × 10^18 > INT_MAX
```

Strings

```
cube("hello")   // ("hello")*("hello")*("hello") – compilation error
```

(b)

```
#define remainder(n) ((n)%4)
```

only work with integar
send Strings cause error
no side effects errors

(C)

```
#define value(x,y) (((x)*(y))>100 ? 1 : 0)
```

Overflow
send Strings cause error
no side effects errors

2- Write a macro NELEMS(a) that computes the number of elements in a one-dimensional array a. Hint: See the discussion of the sizeof operator in Section 8.1.

answer:

```
#define  NELEMS(a) ((int)(sizeof(a)/sizeof(a[0])))
```

---

3. Let DOUBLE be the following macro:

```
#define DOUBLE(x) 2*x
```

(a) What is the value of DOUBLE(1+2)?
(b) What is the value of 4/DOUBLE(2)?
(c) Fix the definition of DOUBLE.

answer:

(a)

```
  DOUBLE(1+2) = 2* 1 + 2 =4
```

(b)

```
  4/ DOUBLE(2)  = 4/2*2 = 4
```

(C)

```
#define DOUBLE(x) (2*(x))
```

For each of the following macros, give an example that illustrates a problem with the macro and show how to fix it.

```
(a) #define AVG(x,y) (x+y)/2
(b) #define AREA(x,y) (x)*(y)
```

answer:

(a)

```
#define AVG(x,y) (x+y)/2
float x = 20/AVG(5,5)  --> 20/(5+5)/2 = 1
//to fix it
#define AVG(x,y) (((x)+(y))/2)
float x = 20/AVG(5,5)  --> 20/((5+5)/2) = 4
```

(b)

```
(b) #define AREA(x,y) (x)*(y)
float x = 5/AREA(5,5)  --> 5/5*5= 5
//to fix it
(b) #define AREA(x,y) ((x)*(y))
float x = 5/AREA(5,5)  -->5/(5*5)= .2
```

---

*5. Let TOUPPER be the following macro:

```
#define TOUPPER(c) ('a'<=(c)&&(c)<='z'?(c)-'a'+'A':(c))
```

Let s be a string and let i be an int variable. Show the output produced by each of the following program fragments.

```
(a)
strcpy(s, "abcd"); i = 0; putchar(TOUPPER(s[++i]));
(b)
strcpy(s, "0123"); i = 0; putchar(TOUPPER(s[++i]));
```

answer:

(a)

```
#define TOUPPER(c) ('a'<=(c)&&(c)<='z'?(c)-'a'+'A':(c))
strcpy(s, "abcd");//s="abcd"
i = 0;
putchar(TOUPPER(s[++i]));
//putchar(('a'<=s[(++i)]&&s[(++i)]<='z'?s[(++i)]-'a'+'A':s[(++i)]))
//putchar(('a'<=s[1]&&s[2]<='z'?s[3]-'a'+'A':s[4]))
//putchar(('a'<='b'&&'c'<='z'?'d'-'a'+'A':'\0'))
//answser :'D'
```

(b)

```
#define TOUPPER(c) ('a'<=(c)&&(c)<='z'?(c)-'a'+'A':(c))
strcpy(s, "0123");
i = 0;
putchar(TOUPPER(s[++i]));
//putchar(('a'<=s[(++i)]&&s[(++i)]<='z'?s[(++i)]-'a'+'A':s[(++i)]))
//putchar(('a'<=s[1]&&s[++i]<='z'?s[++i]-'a'+'A':s[2]))
//putchar(('a'<='1'&&'1'<='z'?'1'-'a'+'A':'2'))
//answser :'2'
```

6-

(a) Write a macro DISP(f,x) that expands into a call of printf that displays the value of the function f when called with argument x. For example, DISP(sqrt, 3.0); should expand into printf("sqrt(%g) = %g\n", 3.0, sqrt(3.0));
(b) Write a macro DISP2(f,x,y) that's similar to DISP but works for functions with two arguments.

```
DISP(sqrt, 3.0);
printf("sqrt(%g) = %g\n", 3.0, sqrt(3.0)); (
```

answer:

(a)

```
#define DISP(f,g) printf(#sqrt"(%g) = %g\n", g, f(g))
DISP(sqrt, 3.0);
```

(b)

```
#define  DISP2(f,x,y) printf( #f"(%g, %g) = %g\n",x, y, f(x,y))
DISP2(pow, 2, 3);
```

---

7. Let GENERIC_MAX be the following macro:

```
#define GENERIC_MAX(type) \
 type type##_max(type x, type y) \
 { \
  return x > y ? x : y; \
 }
```

a) Show the preprocessor's expansion of GENERIC_MAX(long).
(b) Explain why GENERIC_MAX doesn't work for basic types such as unsigned long.
(c) Describe a technique that would allow us to use GENERIC_MAX with basic types such as unsigned long. Hint: Don't change the definition of GENERIC_MAX

answer:

(a)

```
GENERIC_MAX(long)
long int_max(long x, long y)
{
    return x > y ? x : y;
}
```

(b)

```
GENERIC_MAX(unsigned long);
unsigned long unsigned long_max(unsigned long x, unsigned long y)
{
    return x > y ? x : y;
}
// the type"unsigned" for function will be dublucated
```

(c) to fix it

```
typedef unsigned long unsigned_long;
 GENERIC_MAX(unsigned long)
 unsigned_long unsigned_long_max(unsigned_long x, unsigned_long y)
```

```
{
    return x > y ? x : y;
}
```

---

8. Suppose we want a macro that expands into a string containing the current line number and file name. In other words, we'd like to write

```
const char *str = LINE_FILE;
```

and have it expand into

```
const char *str = "Line 10 of file foo.c";
```

where foo.c is the file containing the program and 10 is the line on which the invocation of LINE_FILE appears. Warning: This exercise is for experts only. Be sure to read the Q&A section carefully before attempting!

answer:

```
#define LINE_T_STRING(x) #x
#define AAR_LINE_T_STRING(x) LINE_T_STRING(x)
#define LINE_FILE "line " LINE_T_STRING(__LINE__)" of file " __file__

const char *str = LINE_FILE;
```

---

9. Write the following parameterized macros.

```
(a) CHECK(x,y,n) - Has the value 1 if both x and y fall between 0 and n - 1,
inclusive.
(b) MEDIAN(x,y,z) - Finds the median of x, y, and z.
(c) POLYNOMIAL(x) - Computes the polynomial 3x 5 + 2x 4 - 5x 3 - x 2 + 7x - 6.
```

answer:

(a)

```
# define CHECK(x,y,n) (((x)>=0)&&((y)>=0)&&((x)<(n))&&((y)<(n)))
```

(b)

```
#define MEDIAN(x,y,z) ((x > y) ? \
        ((x < z) ? x : ((y > z) ? y : z)) : \
        ((y < z) ? y : ((x > z) ? x : z)))
```

(c)

```
#define x_2(X) ((x)*(x))
#define x_3(X) ((x_2(x))*(x))
#define x_4(x) ((x_3(x))*(x))
#define x_5(x) ((x_4(x))*(x))
#define POLYNOMIAL(x)  ((3*(x_5(x)))+(2*(x_4(x)))-(5*(x_3(x)))-(x_2(x))+(7*
(x)) -6)
```

---

10. Functions can often—but not always—be written as parameterized macros. Discuss what characteristics of a function would make it unsuitable as a macro.

answer:

1. Presence of Side Effects (`++x` , `x++`)
2. Recursion
3. Need for Local Variables
4. High Complexity (Long Code Blocks)
5. Requirement for Type Checking

---

11. (C99) C programmers often use the fprintf function to write error messages:

```
fprintf(stderr, "Range error: index = %d\n", index);
```

stderr is C's "standard error" stream; the remaining arguments are the same as those for printf, starting with the format string. Write a macro named ERROR that generates the call of fprintf shown above when given a format string and the items to be displayed:

```
ERROR("Range error: index = %d\n", index);
```

answer:

```
# define ERROR(...) \
                 fprintf(stderr, __VA_ARGS__)

ERROR("Range error: index = %d\n", index);
fprintf(stderr, "Range error: index = %d\n", index);
```

---

12. Suppose that the macro M has been defined as follows:

```
#define M 10
Which of the following tests will fail?
(a) #if M
(b) #ifdef M
(c) #ifndef M
(d) #if defined(M)
(e) #if !defined(M)
```

answer:

(a) TRUE

(b) TRUE

(c) FALSE

(d) TRUE

(e)FALSE

13.

(a) Show what the following program will look like after preprocessing.
You may ignore any lines added to the program as a result of including the header.

```
#include <stdio.h>
#define N 100
void f(void);
int main(void)
{
    f();
#ifdef N
```

```
#undef N
#endif
    return 0;
}
void f(void)
{
#if defined(N)
    printf("N is %d\n", N);
#else
    printf("N is undefined\n");
#endif
}
```

(b) What will be the output of this program?

answer:

(a)

```
void f(void);
int main(void)
{
    f();


    return 0;
}
void f(void)
{



    printf("N is undefined\n");

}
```

(b)

```
N is undefined
```

***14. Show what the following program will look like after preprocessing. Some lines of the program may cause compilation errors; find all such errors.**

```c
#define N = 10 //n 10 not use =
#define INC(x) x+1 //INC should have((x)+1)
#define SUB (x,y) x-y // SUB(x,y) without space
#define SQR(x) ((x)*(x))
#define CUBE(x) (SQR(x)*(x))
#define M1(x,y) x##y
#define M2(x,y) #x #y // should be #x , #y

int main(void)
{
    int a[N], i, j, k, m;
#ifdef N
    i = j;
#else
    j = i;
#endif
    i = 10 * INC(j);
    i = SUB(j, k);
    i = SQR(SQR(j));
    i = CUBE(j);
    i = M1(j, k);
    puts(M2(i, j));
#undef SQR
    i = SQR(j);
#define SQR // sdefine sqr but not contain value and type is simple
    i = SQR(j);
    return 0;
}
```

answer:

```c
int main(void)
```

```
{
    int a[=10], i, j, k, m;

    i = j;



    i = 10 * j+1; // j+1 should have a (j+1)
    i =  (x,y) x-y(j, k);
    i = ((((j)*(j)))*(((j)*(j))));
    i = (((j)*(j))*(j));
    i = jk ;// jk is not define
    puts("i" "j");

    i = SQR(j); //error sqr is undefine

    i = (j);
    return 0;
}
```

---

15. Suppose that a program needs to display messages in either English, French, or Spanish. Using conditional compilation, write a program fragment that displays one of the following three messages, depending on whether or not the specified macro is defined:

```
Insert Disk 1 (if ENGLISH is defined)
Inserez Le Disque 1 (if FRENCH is defined)
Inserte El Disco 1 (if SPANISH is defined)
```

answer:

```
#define ENGLISH
#ifdef ENGLISH
    printf("Insert Disk 1\n");
#elif defined(FRENCH)
    printf("Inserez Le Disque 1 \n");
#elif defined(SPANISH)
    printf("Inserte El Disco 1 \n");
#endif
```

---

16. (C99) Assume that the following macro definitions are in effect:

```
#define IDENT(x) PRAGMA(ident #x)
#define PRAGMA(x) _Pragma(#x)
```

What will the following line look like after macro expansion?
IDENT(foo)

answer:

```
PRAGMA( ident "foo");
_Pragma("ident \"foo\"")
#pragma ident "foo"
```

The end ..